Regularisation and the bias–variance trade-off

Why more capacity stops helping, and what to do about it

LECTURE 5 | GÉRON CH. 4

One identity, three terms, and the one line that repairs an ill-posed fit

---

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Worked example: the Titanic passenger manifest

Where we are

Lecture 4 derived gradient descent — the gradient, the update, the bound $\eta < 2/\lambda_{\max}$, scaling through the condition number, the one-row unbiased gradient — and used it to fit a logistic regression to 712 Titanic passengers: log-odds, a log loss that is a proper scoring rule, a rank-deficient design repaired, calibration, softmax. Then it added polynomial capacity and recorded **both** curves:

Degree	Columns	Training log loss	Held-out log loss
1	22	0.395	**0.468 ✓**
2	32	0.381	0.471
3	52	0.355	0.539
5	143	0.286	**✗ 1.952**

One curve falls forever; the other turns and climbs. Two questions it cannot answer: **where does the gap between them come from**, and **how far down could 0.468 ever go**, against an anchor of 0.666? They have opposite repairs.

Today

- **The mathematics** — the bias–variance decomposition: what a generalisation error is made of
- **Measuring it** — the noise floor, the three terms on our own pipeline, learning and validation curves
- **What went wrong at degree 5** — three faults, and what adding $\alpha\mathbf{I}$ does to each
- **Regularisation** — ridge, lasso, early stopping
- **Choosing α honestly**, and the test set once

Open Lecture 5 in Colab

The decomposition and what each expectation is over; reading a diagnosis off a pair of curves; ridge and lasso and what each does to the coefficients. Solver names, wall-clock timings, and the Titanic feature engineering.

EXAMINABLE

NOT EXAMINABLE

First: what is random here?

Not the passenger. Fix one passenger x and ask about them repeatedly.

- The **training set** D is random — you happened to get these 712 rows out of the 891
- The **label** y of the passenger is random — two people with identical recorded values did not both survive

Every expectation in this derivation is over those two sources and nothing else. The model is deterministic given D.

The quantity to decompose

$$\mathbb{E}_{D,\,y}\left[\left(y - \hat{p}_D(x)\right)^2\right]$$

EXPECTED SQUARED ERROR AT ONE PASSENGER

Symbol	Meaning	Observed?
$\hat{p}_D(x)$	what *your* model predicts, having seen D	**yes ✓**
$\bar{p}(x) = \mathbb{E}_D[\hat{p}_D(x)]$	the average prediction over all training sets you could have drawn	**no — estimated ✗**
$p^*(x)$	the true probability that this passenger survives	**never ✗**

Averaged over passengers this is the **Brier score** the previous lecture already reported. We decompose it at a single x and then average — nothing is lost by doing it one passenger at a time.

The decomposition

$$\mathbb{E}\left[(y - \hat{p}_D)^2\right] = (p^* - \bar{p})^2 + \mathbb{E}_D\left[(\hat{p}_D - \bar{p})^2\right] + p^*(1 - p^*$$

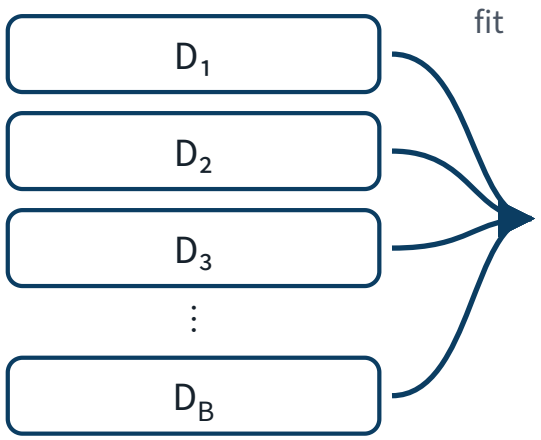
Term	Means	Cured by
squared bias	wrong *on average* — the model family cannot reach the truth	more capacity
variance	unstable — a different sample gives a different answer	less capacity, or more data
noise	the label is not a function of the recorded columns	✗ **nothing**

How it arises: inside the square, add and subtract $\bar{p}$ and average over D; then add and subtract p^* and average over y. Both cross terms vanish, because $\bar{p} = \mathbb{E}_D[\hat{p}_D]$ and $p^* = \mathbb{E}_y[y]$; the last term is $\mathrm{Var}(y)$ for a Bernoulli label.

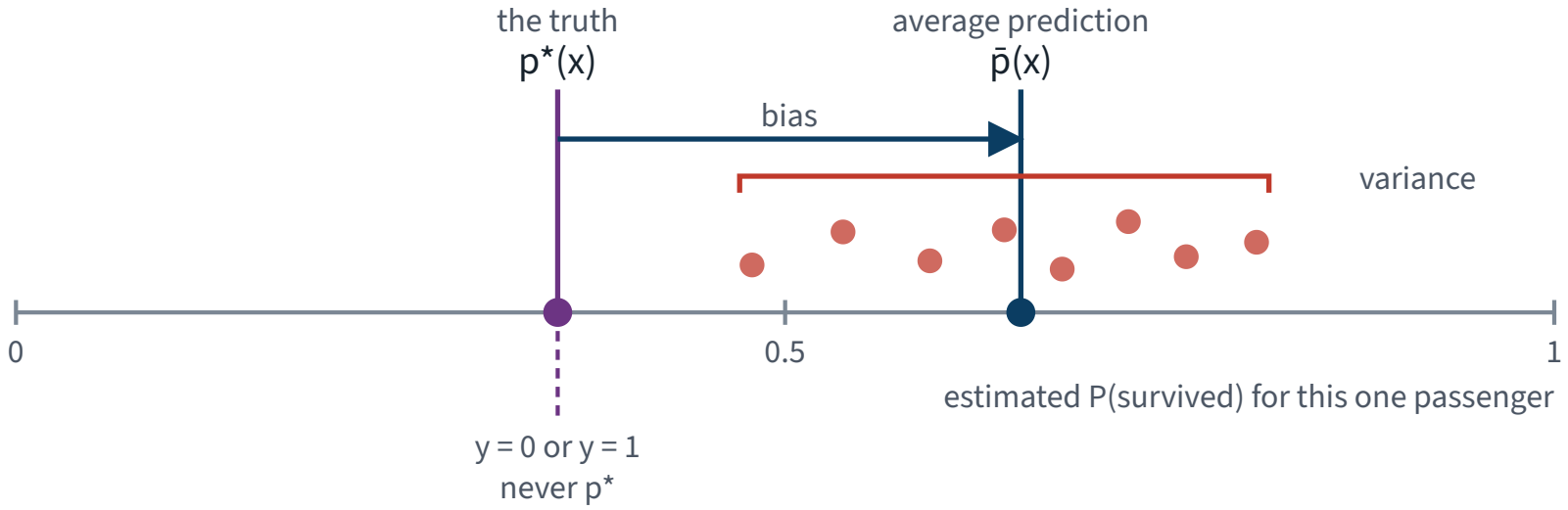
✗ The first two repairs point in opposite directions. That is the whole difficulty, and it is why it is called a trade-off.

The three terms, in one picture

One passenger, every training set you could have drawn



712 rows — but which 712?



$$E\,[(y - \hat{p})^2] = \mathbf{(p^* - \bar{p})^2} + \mathbf{E\,[(\hat{p} - \bar{p})^2]} + \mathbf{p^*(1 - p^*)}$$

total error — squared bias — wrong on average — variance — unstable — noise — nothing to be done

Bias is the distance from the centre of the spread to the truth. Variance is the width of the spread. Neither is visible from one training set.

An honest caveat before we measure anything

The identity is exact for **squared error**. Our headline metric is log loss, and log loss does *not* split into three additive terms like this.

> **WHAT WE DO ABOUT IT**
>
> The decomposition is measured on the **Brier score** — which the previous lecture already reported alongside log loss, and which is the squared error of a probability. The *shape* of the answer transfers; the numbers are Brier numbers and are labelled as such.

Saying “bias–variance” about a log loss curve is common and slightly wrong. Doing it knowingly, and saying which metric you decomposed, is the difference.

What the derivation buys you

The result	What it lets you do
Three additive terms	ask *which* one is large before choosing a repair
Variance does not mention p^*	estimate it without knowing the truth — refit and look at the spread
Noise depends only on x	measure a **floor** that no model of these columns can beat
✓ The gap between two curves	**✓ read a diagnosis off a plot in ten seconds**

✓ The rest of this lecture is that table, executed: measure all three terms on the pipeline of the previous lecture, then repair the one that turned out to be large.

Start with the term you cannot fix

p^* is never observed — but where several passengers share *exactly* the same recorded values, every model of those columns must give them the same number. Whatever their outcomes do inside that cell is a floor.

```
keys  = ["Sex", "Pclass", "AgeBand", "FamBand", "Embarked"]
cells = full.groupby(keys, observed=True)["Survived"].agg(["sum", "count"])

# k(m-k)/(m(m-1)) is the unbiased estimator of p(1-p) from m Bernoulli draws
multi = cells[cells["count"] >= 2]
var   = multi["sum"] * (multi["count"] - multi["sum"]) / (
        multi["count"] * (multi["count"] - 1))
noise = (var * multi["count"]).sum() / multi["count"].sum()
```

133 distinct cells; 102 of them hold at least two passengers, covering 858 people.

The floor

0.121

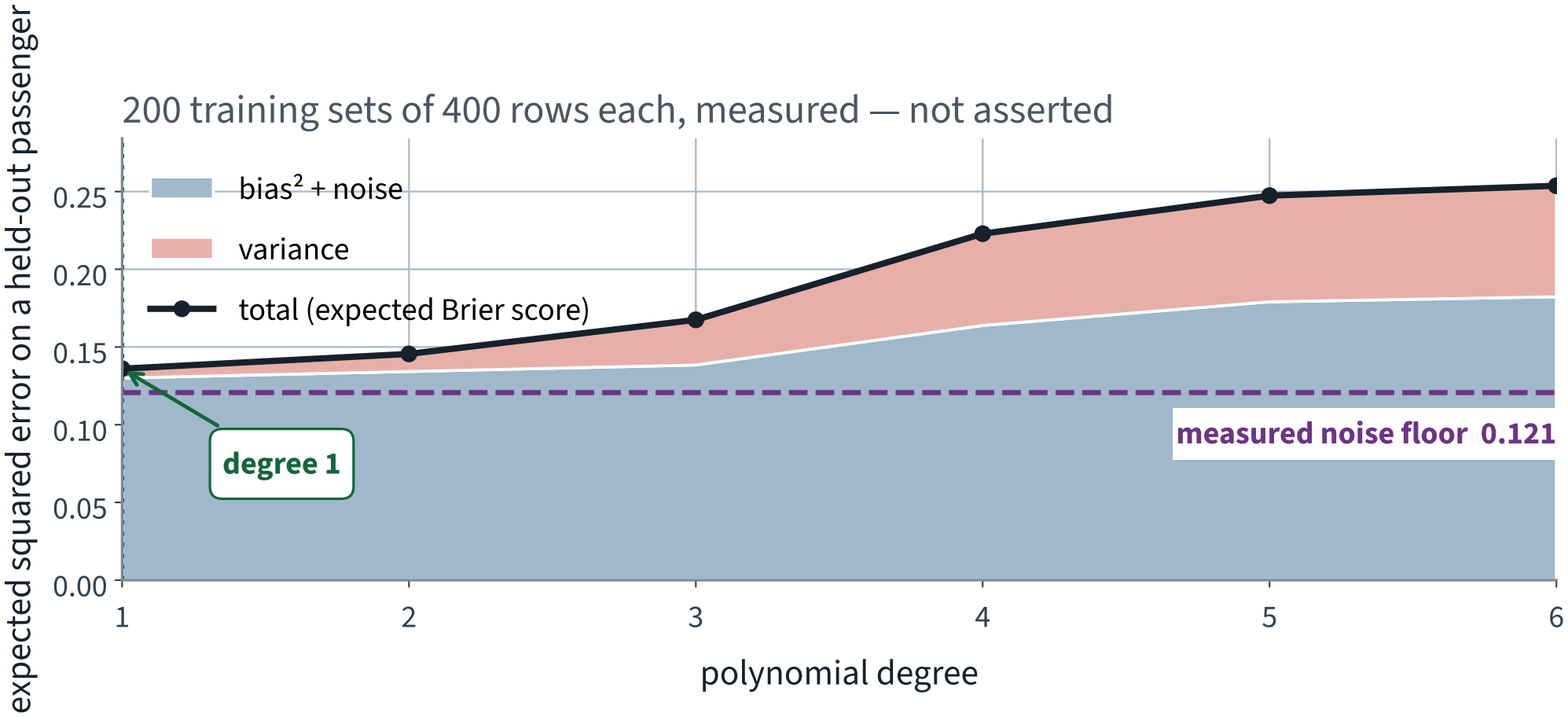
Brier score. Measured, not assumed.

56 of the 102 shared cells are *mixed* — 657 passengers sit in a cell where the outcome went both ways. That is 74% of everyone on board, living inside a cell whose answer cannot be certain.

It is an *under*-estimate of the true noise: two passengers in the same cell may still differ in ways the manifest never recorded. The real floor is at least this high.

> **FAILURE CONDITION — A MODEL MEASURED AGAINST ZERO ERROR**
>
> The irreducible noise here is 0.121 Brier, measured from passengers whose recorded values are identical and whose outcomes differ. A model judged against zero rather than against that floor will be tuned forever toward an error that no model of these columns can reach.

The degree sweep, taken apart



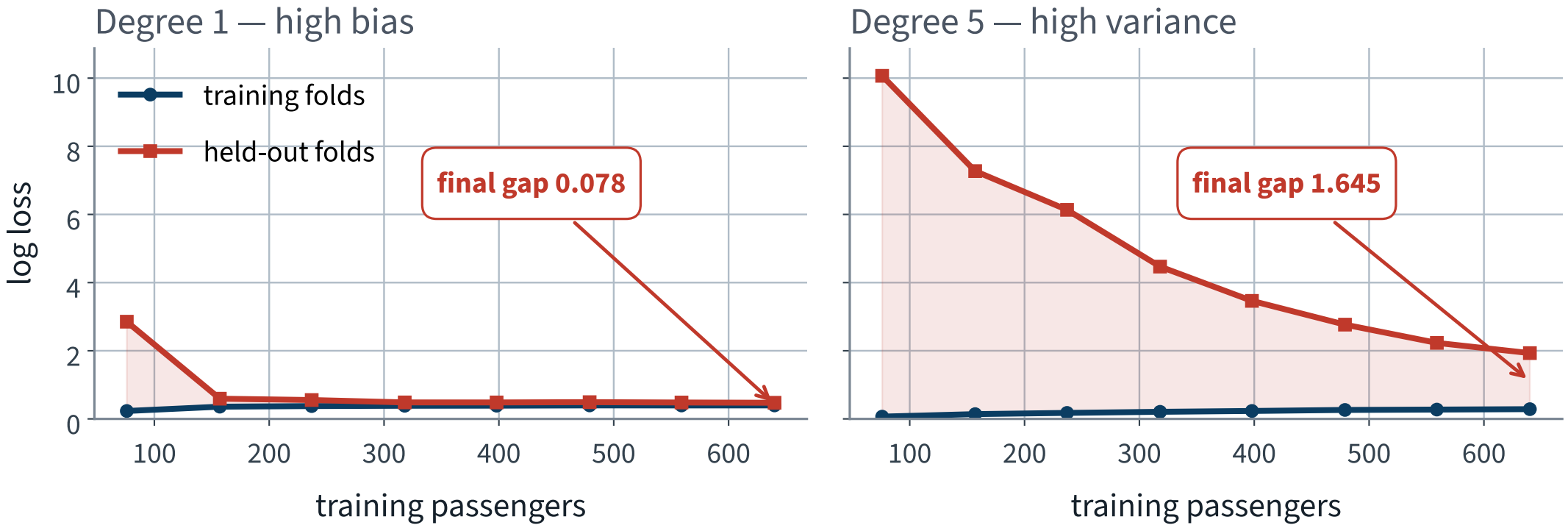
200 training sets of 400 rows drawn from the 712, each scored on the same 179 held-out passengers. From degree 1 to 6 the variance is multiplied by **twelve**; everything else is nearly flat.

Two shapes, two diagnoses

- Both curves plateau **high and together** → bias. The model is too rigid; more data changes nothing
- A gap that is **still open** at the last point → variance. More data would help, and you may not be able to get any

These are the only two shapes, and telling them apart takes ten seconds once you know what you are looking at.

Degree 1 and degree 5, as the data grows



A *learning curve*: the x-axis is training rows. Left: the curves have met. Right: they have not, and the gap is still falling — the model is starved of rows, not of capacity.

The other curve: score against a *hyperparameter*

Curve	x-axis	Answers
Learning curve	training-set size	would more *rows* help?
Validation curve	one hyperparameter	which *setting* is right, and is the optimum interior?

```
from sklearn.model_selection import validation_curve

train, valid = validation_curve(
    pipeline(degree=5), X_train, y_train, cv=cv,
    param_name="clf__C", param_range=np.logspace(-4, 4, 17),
    scoring="neg_log_loss")
```

✗ Read the *ends* of a validation curve first. A minimum sitting on the edge of the range means the range was too narrow and the answer you read off it is an artefact of where you stopped looking.

What went wrong at degree 5: the suspect

1.952

Degree 5 is **almost three times worse than saying nothing at all**.

Not worse than the best model. Worse than the anchor of 0.666 — worse than a model that has learned only how many people died.

Diagnosis 1 · Variance, straight off the decomposition

	Degree 1	Degree 5
Columns	22	✗ 143
Training rows	712	712
Rows per weight	32	✗ 5
Variance term, Brier	0.006	✗ 0.068

Five rows per weight. Move one passenger between folds and the fitted surface moves with them.

This is the diagnosis the derivation was built to deliver, and it is not yet the whole story. Two more faults are underneath it.

Diagnosis 2 · The minimiser does not exist

Suppose that in the expanded feature space some $\boldsymbol{\theta}$ separates the classes perfectly. Then

$$\sigma\left(c\,\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}\right) \longrightarrow \begin{cases} 1 & \text{if } \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x} > 0 \\ 0 & \text{if } \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x} < 0 \end{cases} \qquad \text{as } c \to \infty$$

so the training log loss falls monotonically towards zero as $\|\boldsymbol{\theta}\| \to \infty$. The infimum is never attained.

Degree	1	2	3	4	5	6
Converged	yes ✓	yes ✓	yes ✓	✗ no	✗ no	✗ no
Largest $\lvert\theta_j\rvert$	4.87	5.65	6.51	✗ 19.51	✗ 8.20	4.27

Diagnosis 3 · The minimiser is not unique

Recall from the previous lecture that `FamilySize = SibSp + Parch + 1` exactly, on every row. Standardise the five numeric columns, form $\mathbf{X}^\intercal\mathbf{X}$, and look at its spectrum:

```
>>> np.linalg.eigvalsh(Z.T @ Z)
array([5.59e-13, 4.29e+02, 5.43e+02, 7.90e+02, 1.80e+03])
```

The smallest eigenvalue is zero to the precision the arithmetic allows. Four columns of information in five columns of data.

✗ Condition number about 2.82×10^{15}. A double carries about 10^{16} of relative precision, so roughly *one* significant digit survives the solve.

What adding $\alpha\mathbf{I}$ does to that spectrum

If $\mathbf{X}^\intercal\mathbf{X}\,\mathbf{v} = \lambda\mathbf{v}$, then

$$(\mathbf{X}^\intercal\mathbf{X} + \alpha\mathbf{I})\,\mathbf{v} \;=\; (\lambda + \alpha)\,\mathbf{v}$$

- Same eigenvectors; every eigenvalue moved up by exactly α
- $\mathbf{X}^\intercal\mathbf{X}$ is positive semi-definite, so $\lambda \geq 0$, so $\lambda + \alpha \geq \alpha > 0$
- No eigenvalue is zero, so **the matrix is invertible for every $\alpha > 0$**

THE NORMAL EQUATION, REPAIRED

Lecture 3 derived $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$ and showed that it fails when $\mathbf{X}^\intercal\mathbf{X}$ is singular. **Adding $\alpha\mathbf{I}$ makes it exist, for every $\alpha > 0$, whatever the columns do.** Those three lines are the proof.

and it bounds the conditioning

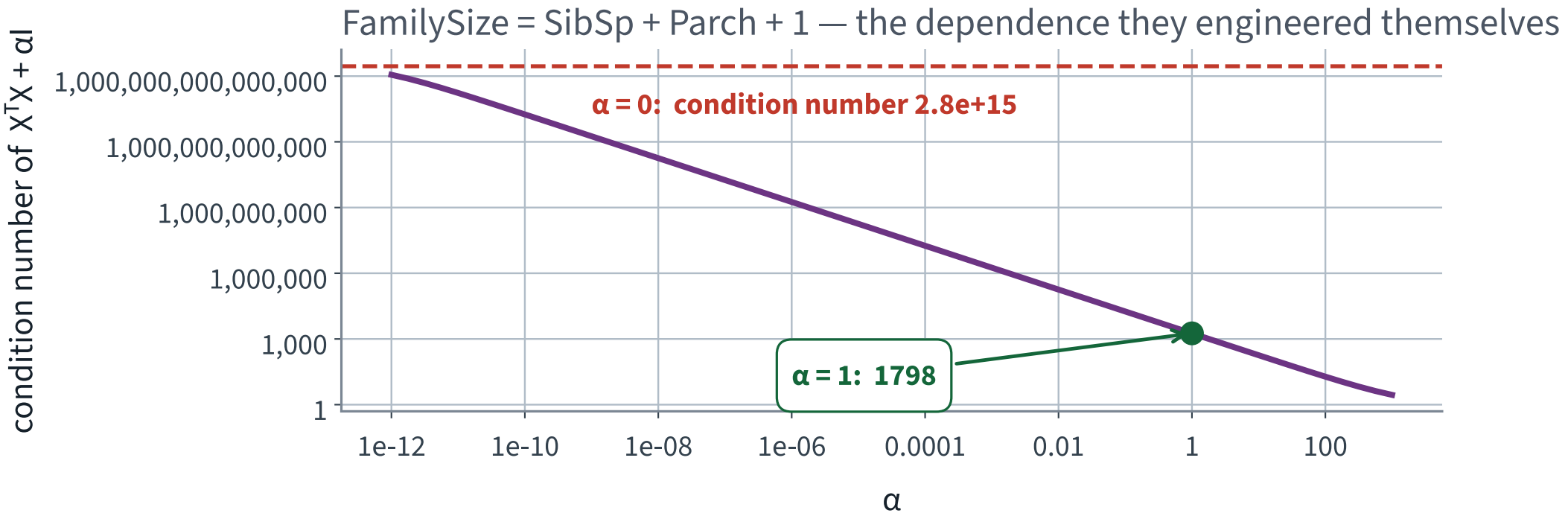
$$\kappa(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}) = \frac{\lambda_{\max} + \alpha}{\lambda_{\min} + \alpha} \leq \frac{\lambda_{\max} + \alpha}{\alpha}$$

The bound on the right does not mention $\lambda_{\min}$ at all. Whatever the data do — exact collinearity included — the conditioning is controlled by a number you choose.

✓ At $\alpha = 1$ the condition number of our singular matrix falls from 2.82×10^{15} to 1798.

And κ is what set the step count for gradient descent in the previous lecture. The penalty is not only a statistical device; it is what makes the optimisation tractable.

Condition number against α



A straight line of slope -1 over twelve decades: in this regime the condition number is simply $\lambda_{\max}/\alpha$. The dependence in the data is not repaired — it is *dominated*.

The idea, in one sentence

Stop asking for the best fit. Ask for the best fit *among small models*.

$$J(\boldsymbol{\theta}) = \underbrace{L(\boldsymbol{\theta})}_{\text{fit the data}} + \underbrace{\alpha\,\Omega(\boldsymbol{\theta})}_{\text{stay small}}$$

Ω penalises the size of the weights; α says how much you care. Setting $\alpha = 0$ recovers exactly the previous lecture.

✓ Constraining the model is one of the two cures for variance, and the only one available when the dataset is fixed at 891 rows.

Ridge — the ℓ_2 penalty

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{j=1}^{n} \theta_j^2$$

- The sum starts at $j = 1$: **the bias term θ_0 is not penalised**. Shrinking the intercept would pull every prediction towards zero rather than towards the mean
- Equivalently $\Omega(\boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{w}\|_2^2$, where $\mathbf{w}$ is $\boldsymbol{\theta}$ without the bias

Also called Tikhonov regularisation; the book gives both names.

Lasso — the ℓ_1 penalty

$$J(\boldsymbol{\theta}) \;=\; \mathrm{MSE}(\boldsymbol{\theta}) \;+\; 2\alpha \sum_{j=1}^{n} |\theta_j|$$

Least Absolute Shrinkage and Selection Operator. The word **selection** is the point: lasso drives weights to exactly zero, so it performs feature selection as a side effect of fitting.

Ridge shrinks everything and eliminates nothing.

Why ℓ_1 produces exact zeros and ℓ_2 does not

Look at the gradient of the penalty near $\theta_j = 0$.

Ridge

$$\frac{\partial}{\partial \theta_j}\,\theta_j^2 = 2\theta_j \;\to\; 0$$

The push towards zero fades as you approach zero. You arrive only in the limit.

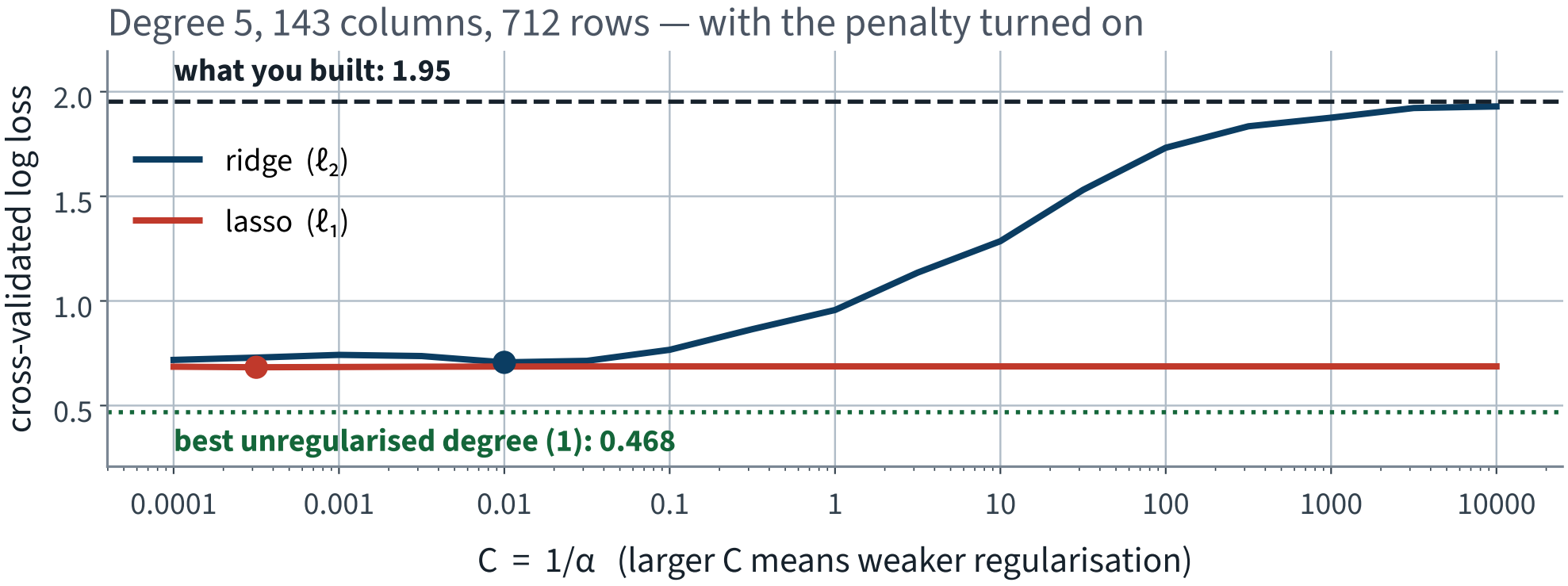
Lasso

$$\frac{\partial}{\partial \theta_j}\,|\theta_j| = \pm 1$$

The push keeps its size right up to zero. If the data pulls with less than α, the weight reaches zero and stops.

At $\theta_j = 0$ the absolute value is not differentiable; the subgradient is the whole interval $[-1, 1]$, and any data pull inside that interval is held at exactly zero.

What the penalty buys at degree 5



Both curves are far below the dashed line. Neither of them reaches the dotted one.

And now the uncomfortable part

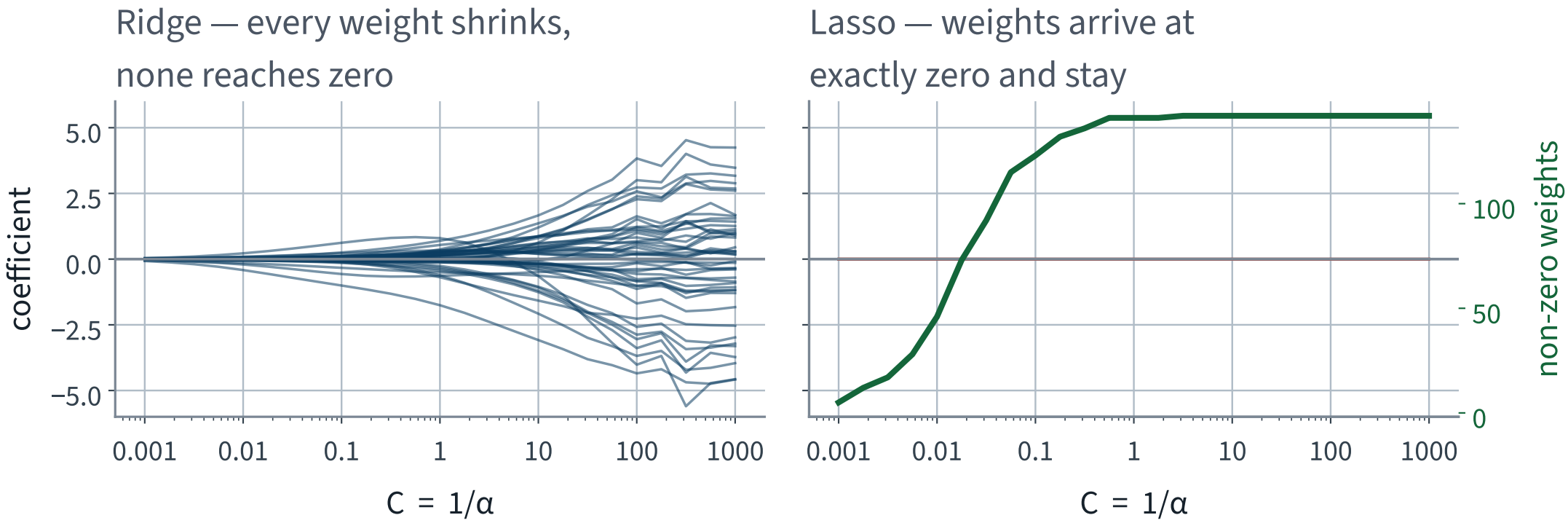
Model	Cross-validated log loss
Degree 5, lasso, tuned	0.683
Degree 1, no penalty at all	**0.468 ✓**
Degree 2, no penalty at all	**0.471 ✓**

✗ The best regularised degree-5 model beats neither of the plain low-degree models we already had.

Degrees 1 and 2 differ by 0.003, inside a fold spread of 0.13: they are tied, so both rows are marked.

Regularisation repaired an over-complex model. It did not turn it into a better one than the simple model, because there was never much bias to buy back — the decomposition said so, and this is what that sentence looks like in a table.

Ridge shrinks; lasso selects



Left: every path approaches zero and none arrives. Right: paths sit at exactly zero and leave one at a time as the penalty weakens.

Early stopping — a penalty by another route

Fit iteratively, and **stop before the optimiser gets where it is going**.

- Gradient descent starts at $\boldsymbol{\theta} = \mathbf{0}$ — the smallest possible model
- Each epoch moves the weights outward, fitting more of the training set
- Held-out error falls, reaches a minimum, and then rises

✓ Géron calls it a "beautiful free lunch": no penalty term and no extra term in the objective — just a stopping rule. The knob is the epoch count, and it is a hyperparameter like any other.

Choosing a hyperparameter is choosing a model

```
best_C, best_score = None, np.inf

for C in np.logspace(-4, 4, 17):
    m = pipeline(degree=3, C=C).fit(X_train, y_train)
    score = log_loss(y_test, m.predict_proba(X_test)[:, 1])
    if score < best_score:
        best_C, best_score = C, score
```

FAILURE CONDITION — A HYPERPARAMETER CHOSEN ON THE ROWS YOU THEN REPORT

`best_score` is a **minimum over seventeen noisy estimates**, and the minimum of noisy estimates is biased downward even when every estimate is individually unbiased. The code never writes to the test set. It *reads* it seventeen times, and reading is enough.

Measure it rather than asserting it

20 splits. On each: run the loop above, and also the honest version that picks `C` by 5-fold cross-validation *inside* the training part. Score both on the same held-out rows.

Reported number	Log loss	Spread over 20 splits
Chosen on the test set, scored on the test set	✗ **0.451**	0.049
Chosen by cross-validation, scored on the test set	**0.470** ✓	0.045

Optimism: 0.019 of log loss, about 4.1%. On 13 splits of 20 the first number is the flattering one, and the two procedures picked the same `C` on 7 splits of 20.

The procedure that does not have the problem

```
gs = GridSearchCV(pipeline(degree=3), {"clf__C": np.logspace(-4, 4, 17)},
                  scoring="neg_log_loss", cv=cv).fit(X_train, y_train)

print(-gs.best_score_)                                  # the honest estimate
print(log_loss(y_test, gs.predict_proba(X_test)[:, 1]))   # the test set, once
```

Same seventeen candidates, same runtime, **two numbers instead of one**. Report both. The second is allowed to be worse than the first, and if it is much worse that is itself the finding.

The standing rule, extended by eleven words: nothing derived from the test set may appear in the training path — including the choice of any hyperparameter.

The honest numbers

The 179 test passengers, untouched since the split; every candidate's hyperparameters fixed by cross-validation before they were scored.

Model	Log loss	Brier	Accuracy
Degree 5, no penalty — where the sweep ended	**✗ 1.472**	0.182	76.0%
Degree 5, ridge, tuned	0.804	0.173	77.1%
Degree 5, lasso, tuned	0.677	0.244	65.9%
Degree 1, scikit-learn defaults	**0.433 ✓**	**0.134 ✓**	**82.7% ✓**
Degree 2, no penalty — **the winner**	**0.426 ✓**	**0.134 ✓**	**82.7% ✓**

The winner improves on the degree-5 model the previous lecture finished with by **1.046** of log loss.

The winner is a model with no repair in it

Degree 2, no penalty: one of the two degrees tied at the bottom of the held-out curve.

Row four deserves as much attention: `LogisticRegression()` with every default left alone — degree 1, `C=1.0`, `penalty="l2"` — scores 0.433. Statistically it is the same model as the winner.

> **THE RESULT NOBODY WANTED**
>
> A whole lecture of penalties, and the best system on the test set is a two-line model anyone could have written before the previous lecture started.

✓ Report it anyway. The alternative is choosing the interesting model over the better one.

The notebook for this lecture

Open Lecture 5 in Colab

- **Runs on free CPU**
- It rebuilds the degree sweep from scratch, so you can reproduce 1.952 before repairing it — then measures the noise floor and the three terms, and checks that they add up
- Then the learning curves, the three faults at degree 5, the penalty sweeps, the coefficient paths, and choosing α honestly with the test set used once

> **WHAT TO CHANGE**
>
> In the penalty sweep, widen the saga grid `CS_SHORT` to `np.logspace(-8, 4, 25)`. Does the lasso minimum move? If it does, the minimum you read was an artefact of where the grid stopped — read the *ends* of a validation curve first.

Summary

1. Expected squared error splits exactly into squared bias, variance and irreducible noise — over the training set and the label, not over test points
2. A gap still open between training and held-out curves means variance; curves plateauing together mean bias
3. The noise floor is measurable: 0.121 Brier, from passengers whose recorded values are identical
4. Degree 5 failed three ways at once — high variance, no minimiser, and no unique minimiser
5. Adding $\alpha\mathbf{I}$ makes $\mathbf{X}^\top\mathbf{X}$ invertible for every $\alpha > 0$ and bounds its condition number
6. Ridge shrinks, lasso selects, early stopping keeps the weights small by not arriving
7. None of them beat choosing the right capacity. A penalty is a repair, not an upgrade
8. A hyperparameter chosen on the test set costs 0.019 of log loss here, and the cost grows with the size of the search

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 5

five, in the style of the written paper

Exercises — Lecture 5

1 Write the bias–variance decomposition of the expected squared error, and name the one term no model can reduce. [4]

2 A learning curve shows training and validation error both high and close together, and flat as data is added. Diagnose it, and say whether more data would help. [5]

3 Ridge adds αI to $X^{\mathsf{T}}X$ before inverting. Give the numerical consequence, in terms of the condition number. [4]

Solutions on Lecture 6's deck.

Exercises — Lecture 5 (continued)

4 Lasso sets some weights to exactly zero; ridge shrinks every weight but sets none to zero. Explain the difference from the gradient of each penalty near $\theta_j = 0$. [4]

5 You tune α over a grid of seventeen values by scoring each on the test set, then report the best test score as your estimate of future performance. Name the error, and quantify its direction. [5]

Solutions on Lecture 6's deck.

THE FIVE SET AT THE END OF LECTURE 4

Solutions Lecture 4

not part of this lecture

Solutions — Lecture 4

1. Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on…

✓ Unscaled features make the contours elongated, so the gradient points across the valley rather than along it.

- The Hessian is $X^\top X$, so the curvature along a feature goes as the *square* of its scale — multiply a column by ten and its curvature rises a hundredfold, which is how one column comes to dominate
- The learning rate must then suit the steepest direction, which makes it far too small for the shallow one

2. On the Titanic manifest, your logistic regression on one-hot encoded columns produces coefficients that look…

✓ Five come from the encoder, one from you. The repair is both: drop one level per categorical, and delete the engineered column.

- Each one-hot block sums to 1 in every row, and so does the intercept column — five categoricals, five exact dependencies
- The sixth is `FamilySize = SibSp + Parch + 1`, an identity rather than a correlation. Dropping a level alone still leaves the rank one short of the column count, because the dependency you wrote yourself survives it

Solutions — Lecture 4 (2)

3. A model reaches 0.80 accuracy and is badly calibrated. Explain what that means, and name a decision it would…

✓ Its predicted probabilities do not match observed frequencies; any decision using a cost-weighted threshold is then wrong.

- Accuracy reads only the arg-max, so it is blind to the probability that produced it
- Choosing a threshold from expected cost requires the probability to mean what it says

4. In a logistic regression with first class as the reference level, the coefficient on Pclass_3 is -1.169….

✓ It multiplies the *odds* of survival by $e^{-1.169} = 0.31$, relative to first class, holding the other features fixed.

- The linear part predicts the log-odds, so adding 1 to an indicator adds its coefficient to the log-odds and multiplies the odds by its exponential
- The probability moves by an amount that depends on where it started: odds of 1 times 0.31 give $p = 0.24$, while $p = 0.9$ becomes 0.74

Solutions — Lecture 4 (3)

5. State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent…

✓ $\theta \leftarrow \theta - \eta \nabla J$. Only the set the gradient is computed over changes; the rule does not.

- SGD estimates the same gradient from one instance rather than all of them
- The estimate is unbiased — averaged over a uniformly drawn row it is exactly the batch gradient — but noisy, and it costs $O(n)$ per step instead of $O(mn)$

NEXT LECTURE

Decision trees

A model that is not linear in anything and needs no scaling — whose capacity is controlled not by a penalty but by limiting its depth and its leaves: today’s trade-off, with a different dial